Task 1

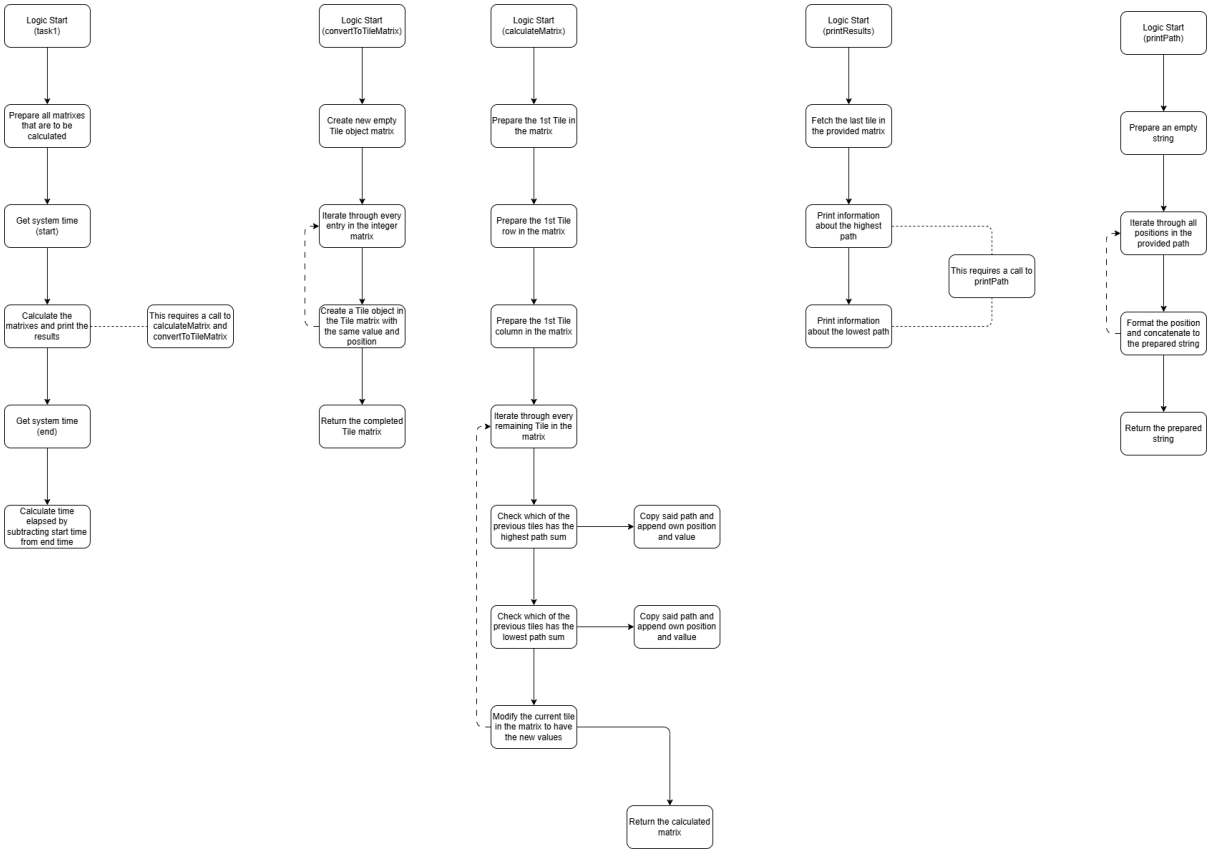
The main purpose of the task 1 application is to find the path through a 2D matrix (whilst only moving down or right) with the highest and the lowest sum of the numbers that said path pass through.

The application does this by calculating the earlier mentioned paths to each individual tile, working its way from the top left to the bottom right. By saving the information to a separate matrix composed to objects designed to house all the required information, the application will not have to calculate every single possible path.

When calculating the path to a specific tile the application simply checks the two previous tiles (the one above the tile and the one to the left of it). With the exceptions being the following:

- First tile in the matrix (doesn't have a path to reach itself)
- First row and column (only have one available path)

This way of calculating the matrixes mean that the application will only have to perform as many calculations as there are tiles, instead of having to check every single possible path individually.



Task 2

The main purpose of task 2 application is to move through a matrix where every row has one more number than the last one. When moving through this matrix the application can only move to a number in the next row which is either the same index or a neighbouring index as the index of the number the application is currently on.

This application uses a method nearly identical to the 1st task, just that the application checks the three tiles of the previous row with the same index as itself or the neighbouring indexes, in the end the application goes through all the tiles in the last row because the task doesn't have a specific tile that it ends on.

In the end the path with the highest and the one with the lowest sum is printed out in the console for the user.

Task 3

The main purpose of the task 3 application is to check if a provided maze is possible to complete or not. (1s act as a walkable tile and 0s act as a wall). For the maze to be possible there has to be a path from the upper left corner to the bottom right corner which is not obstructed by a wall.

The application checks this by using a "spreading" method. It does this by executing a method on the first tile. This method with then mark the active tile as checked to prevent it from being checked again and will then check if any of its neighbouring tiles are not previously checked and are not walls. If this is true, then the tile will execute the same method on said neighbouring tile.

By doing this the application will mark every walkable tile connected to the start tile as having been checked and after all the checking methods are done then it simply checks if the ending tile has been checked. If it hasn't then it isn't connected to the starting tile.